

Plan Global du Projet : GreenTech Intelligence

Ce document constitue la feuille de route centrale du projet. Il commence par l'étape critique : la mise en place de l'environnement technique complet (Phase 0).

 Plan Global du Projet : GreenTech Intelligence	0
 ÉTAPE 1 : Guide d'Installation & Configuration (Checklist Exhaustive)	2
1. Système & Matériel (Le Socle).....	2
2. Outils de Développement Généraux.....	2
3. Environnement Python & Gestion de Paquets.....	2
4. Stack Data (Collecte & Stockage).....	2
5. Stack Intelligence Artificielle (Blocs E2 & E3).....	3
6. Stack Backend & API (Bloc E4).....	3
7. Stack Frontend (Application Client).....	3
8. Documentation.....	3
9. Infrastructure & DevOps (Docker & SaaS).....	4
10. Outils de Veille & Méthodologie.....	4
 ÉTAPE 2 : Data Factory & Gestion de Données (Bloc E1)	5
2.1 Conception & Conformité (Avant de coder).....	5
2.2 Infrastructure de Stockage.....	5
2.3 Programmation de la Collecte (Extraction).....	5
2.4 Traitement Big Data & Nettoyage (Apache Spark).....	6
2.5 Mise à disposition structurée (SQL).....	7
 ÉTAPE 3 : Intelligence Artificielle - Blocs E2 & E3	8
3.1 Veille Technologique & Benchmark (Bloc E2).....	8
3.2 Préparation des Données & MLOps (Data Ops).....	8
3.3 Entraînement & Compétition des Modèles (Sur PC Fixe - ROCm).....	8
3.4 Validation & Packaging (Qualité Modèle).....	9
3.5 Déploiement MLOps (Monitoring).....	9
 ÉTAPE 4 : Backend & API (Blocs E1 & E4)	11
4.1 Conception de l'Architecture API (Design).....	11
4.2 Développement du Serveur API (FastAPI).....	11
4.3 Implémentation de la Sécurité (Auth).....	11
4.4 Exposition des Données (Bloc E1 - C5).....	12
4.5 Intégration de l'IA dans l'API (Bloc E3 - C9/C10).....	12
4.6 Documentation & Tests d'Intégration.....	12
 ÉTAPE 5 : Frontend & Application (Bloc E4)	13

5.1 Initialisation de l'environnement Frontend (React & Vite).....	13
5.2 Développement des Composants UI (Design System).....	13
5.3 Développement des Pages & Parcours Utilisateur.....	13
5.4 Intégration de la Logique Métier (API & State).....	14
5.5 Tests & Accessibilité (Bloc E4 - A8).....	14
 ÉTAPE 6 : DevOps, Déploiement & Maintenance (Bloc E5).....	15
6.1 Automatisation des Tests (CI Pipeline).....	15
6.2 Conteneurisation & Packaging (Docker).....	15
6.3 Livraison Continue (CD Pipeline).....	15
6.4 Monitoring & Observabilité (La Tour de Contrôle).....	16
6.5 Maintenance & Gestion d'Incidents (A9).....	16



ÉTAPE 1 : Guide d'Installation & Configuration (Checklist Exhaustive)

Cette section liste l'intégralité des outils, logiciels et librairies à installer sur vos machines (PC Fixe et Portable) pour respecter scrupuleusement le cahier des charges.

1. Système & Matériel (Le Socle)

- Système d'exploitation : Windows 11 Pro (vérifier les mises à jour).
- Terminal : PowerShell (version la plus récente disponible).
- Accélération Graphique (PC Fixe uniquement) :
 - Drivers AMD Software : Adrenalin Edition à jour.
 - Suite AMD ROCm (HIP SDK) version 6.x ou 7.x compatible Windows pour PyTorch.

2. Outils de Développement Généraux

- Éditeur de code : VSCode (Visual Studio Code).
- Extensions VSCode obligatoires :
 - Python (Microsoft).
 - Ruff (Linting & Formattage).
 - Docker (Gestion des conteneurs).
 - MyST-Parser (Prévisualisation documentation Sphinx).
 - Playwright Test for VSCode.
- Gestionnaire de Version : Git pour Windows (Git SCM).
- Plateforme de Dépôt : Création d'un compte et d'un dépôt "GreenTech-Intelligence" sur GitHub.
- Gestion de Projet : Initialisation d'un projet "Kanban" dans l'onglet Projects de GitHub.

3. Environnement Python & Gestion de Paquets

- Gestionnaire de projet : uv (CLI Astral) - à installer comme remplaçant de pip/poetry.
- Configuration Python : Création d'un environnement virtuel géré par uv.

4. Stack Data (Collecte & Stockage)

Ces librairies doivent être ajoutées au projet Python via uv.

- Requêtes HTTP : httpx (Client HTTP moderne et asynchrone).
- Scraping Framework : scrapy (Framework principal).
- Rendu Navigateur (JS) : playwright (Moteur de navigateur headless) + scrapy-playwright (Connecteur).

- Big Data & Traitement : pyspark (Interface Python pour Apache Spark).
- Base de Données (ORM) :
 - sqlalchemy (ORM version 2.0+).
 - asyncpg (Driver asynchrone pour PostgreSQL).

5. Stack Intelligence Artificielle (Blocs E2 & E3)

- Frameworks de Calcul (PC Fixe - ROCm) :
 - torch (PyTorch) version compatible ROCm (souvent version "preview" ou "nightly" sur Windows).
 - torchvision & torchaudio (versions correspondantes).
- Machine Learning Classique : scikit-learn.
- Modèles & Transformers :
 - transformers (Librairie Hugging Face).
 - huggingface_hub (Pour l'API Serverless et le téléchargement de modèles).
- Tests & Qualité IA : deepchecks (Validation des données et des modèles).
- MLOps & Suivi :
 - mlflow (Tracking des expériences).
 - dvc (Data Version Control) + dvc-s3 (Plugin pour le stockage objet).
 - codecarbon (Mesure de l'empreinte carbone - Green IT).

6. Stack Backend & API (Bloc E4)

- Framework API : fastapi (Framework REST).
- Serveur : uvicorn[standard] (Serveur ASGI).
- Authentification : fastapi-users[sqlalchemy] (Gestion utilisateurs sécurisée).
- Utilitaires :
 - python-multipart (Gestion upload fichiers).
 - loguru (Logging avancé).

7. Stack Frontend (Application Client)

Cet environnement nécessite Node.js (via npm ou bun).

- Build Tool : Vite (Template React TypeScript).
- Framework UI : React.
- Composants & Design :
 - tailwindcss (CSS Utility-first).
 - shadcn-ui (Librairie de composants).
 - lucide-react (Icônes).
- Accessibilité : @axe-core/playwright (Moteur de test d'accessibilité).

8. Documentation

- Générateur : sphinx.

- Parser Markdown : myst-parser.
- Thème Visuel : furo.

9. Infrastructure & DevOps (Docker & SaaS)

- Conteneurisation (Docker Desktop) :
 - Image Docker postgres:15 (Base de données relationnelle).
 - Image Docker minio/minio (Stockage Objet type S3).
 - Image Docker prom/prometheus (Monitoring métriques).
 - Image Docker grafana/grafana (Tableaux de bord).
 - Image Docker grafana/loki (Agrégation de logs).
- Hébergement : Création d'un compte sur Render.
- CI/CD : Configuration des workflows GitHub Actions (fichiers YAML).

10. Outils de Veille & Méthodologie

- Maquettage : Compte sur Penpot (Wireframing).
- Modélisation BDD : Logiciel Looping (MCD/MLD).
- Veille Techno :
 - Compte Inoreader (Agrégateur RSS).
 - Compte Perplexity Pro (Recherche IA).
- Communication : Serveur Discord dédié au projet.

ÉTAPE 2 : Data Factory & Gestion de Données (Bloc E1)

Cette étape couvre la collecte, le nettoyage, et le stockage des données. Elle doit être réalisée après l'installation complète.

2.1 Conception & Conformité (Avant de coder)

- Spécifications Techniques : Rédaction d'un document Markdown listant les contraintes techniques des sources de données (API, Web, Fichiers) et les règles d'extraction.
- Modélisation des Données :
 - Réalisation du Modèle Conceptuel de Données (MCD) sur Looping (Entités : Article, Source, Auteur, Analyse).
 - Génération du Modèle Logique de Données (MLD) et du script SQL correspondant.
- Registre RGPD & Confidentialité :
 - Rédaction du registre des traitements de données (document texte).
 - Définition des procédures de tri pour identifier et anonymiser les éventuelles données personnelles (noms d'auteurs, e-mails).

2.2 Infrastructure de Stockage

- Déploiement Base Relationnelle (PostgreSQL) :
 - Lancement du conteneur Docker PostgreSQL.
 - Création de la base de données greentech_db.
 - Création d'un utilisateur dédié avec droits restreints.
- Déploiement Système Big Data (MinIO - Stockage Objet) :
 - Lancement du conteneur Docker MinIO.
 - Création de deux "Buckets" (compartiments) via l'interface console :
 - raw-data : Pour stocker les fichiers bruts (HTML, JSON non traités).
 - clean-data : Pour stocker les données nettoyées par Spark.

2.3 Programmation de la Collecte (Extraction)

- Module 0 : Configuration Dynamique (Source SQL - PostgreSQL):
 - Création de la table de configuration :
 - Développement d'un script SQL d'initialisation pour la table search_config (Mots-clés, URLs).
 - Insertion des données de référence (ex: "Green IT", "Sustainable AI").
 - Développement du connecteur d'extraction (SQLAlchemy) :
 - Programmation d'une fonction Python get_config_from_db pour exécuter la requête SELECT.
 - Injection dynamique des paramètres récupérés dans les modules API et

Scraping (Validation source SQL).

- **Module 1 : Collecte via API (Bibliothèque httpx) :**
 - Développement d'un script Python pour interroger une API d'actualités technologiques publique.
 - Configuration des requêtes HTTP (headers, timeouts, gestion des erreurs 4xx/5xx).
 - Filtrage initial des données JSON reçues (parsing).
 - Sauvegarde des réponses brutes dans le bucket MinIO raw-data.
- **Module 2 : Scraping Hybride (Bibliothèque scrapy + playwright) :**
 - Développement d'un "Spider" Scrapy ciblant un blog technique moderne (site dynamique).
 - Intégration de Playwright pour charger le JavaScript et effectuer le rendu navigateur des pages.
 - Extraction du contenu HTML complet des articles.
 - Gestion des contraintes techniques (User-Agent, délais entre requêtes pour éviter le blocage).
 - Sauvegarde du HTML brut dans le bucket MinIO raw-data.
- **Module 3 : Ingestion de Fichiers (Script Python) :**
 - Développement d'un script pour lire un dataset historique local (format CSV téléchargé).
 - Programmation de la lecture et du parsing du fichier.
 - Upload des données brutes vers le bucket MinIO raw-data.

2.4 Traitement Big Data & Nettoyage (Apache Spark)

- **Initialisation Spark :** Configuration d'une session PySpark locale connectée à MinIO (via connecteurs S3).
- **Lecture (Extraction Big Data) :**
 - Programmation de la lecture des données brutes depuis le Data Lake MinIO via Spark.
 - (Note : Cette étape valide la compétence "Connexion et extraction depuis un système Big Data").
- **Script de Nettoyage Automatisé :**
 - Programmation de la lecture des fichiers bruts depuis MinIO raw-data.
 - Développement des fonctions de nettoyage :
 - Suppression des balises HTML résiduelles.
 - Identification et suppression des entrées corrompues ou incomplètes.
 - Homogénéisation des formats de dates (ISO 8601) et des encodages textes.
 - Agrégation des trois sources (API, Scraping, Fichier) en un DataFrame unique normalisé.
- **Sauvegarde "Propre" :** Écriture du jeu de données final nettoyé dans le bucket

MinIO clean-data (format Parquet ou JSON).

2.5 Mise à disposition structurée (SQL)

- Script d'Ingestion SQL (SQLAlchemy) :
 - Configuration de la connexion asynchrone à la base PostgreSQL.
 - Développement du script qui lit les données propres depuis MinIO ou le DataFrame Spark final.
 - Insertion des métadonnées structurées (Titre, URL, Date, Auteur, Résumé) dans les tables PostgreSQL.
 - Gestion des conflits (Upsert) pour éviter les doublons.
- Validation & Documentation :
 - Exécution de requêtes SQL de vérification pour confirmer que les données sont bien extraites et stockées.
 - Rédaction de la documentation technique expliquant la logique algorithmique, les choix de nettoyage et les commandes pour lancer les scripts.
 - Push de l'ensemble du code source (scripts, Dockerfiles, docs) sur le dépôt distant GitHub.



ÉTAPE 3 : Intelligence Artificielle - Blocs E2 & E3

Cette étape valide l'intégration de services IA, la veille technologique, et le déploiement MLOps de modèles custom sur matériel spécifique (AMD ROCm).

3.1 Veille Technologique & Benchmark (Bloc E2)

- Configuration de la Veille :
 - Création de flux RSS sur Inoreader ciblant "Green IT", "Sustainable AI", "Model Efficiency".
 - Configuration de tâches de recherche automatique sur Perplexity Pro pour synthétiser les tendances hebdo.
 - Rédaction d'une synthèse mensuelle (Document Markdown).
- Benchmark des Services IA (Étude Comparative) :
 - Rédaction d'un document comparant les solutions de résumés automatiques (OpenAI, Mistral, Hugging Face).
 - Critères d'analyse : Coût, Facilité d'intégration, Impact Carbone, Qualité du résumé.
 - Sélection finale : Hugging Face Serverless Inference API (pour le résumé).
- Intégration du Service SaaS (Script Python) :
 - Développement d'un module Python `summarizer.py` utilisant la librairie `huggingface_hub`.
 - Connexion à l'API Serverless pour envoyer le texte nettoyé des articles.
 - Récupération et stockage du résumé généré dans la base PostgreSQL.

3.2 Préparation des Données & MLOps (Data Ops)

- Création du Dataset d'Or (Golden Dataset) :
 - Sélection manuelle de 200 articles variés depuis la base.
 - Annotation binaire manuelle (Labeling) : "Green IT" (1) ou "Non Green IT" (0).
- Mise en place du Versioning (DVC) :
 - Initialisation de DVC (`dvc init`) dans le projet.
 - Configuration du "Remote" DVC vers le bucket MinIO `clean-data` ou un dossier dédié.
 - Versioning du fichier dataset annoté (`dataset.csv.dvc`).
 - Push des données vers le stockage distant pour synchronisation entre PC Fixe et Portable.

3.3 Entraînement & Compétition des Modèles (Sur PC Fixe - ROCm)

Cette phase utilise spécifiquement le GPU AMD 7900 XTX via ROCm.

- Configuration de l'Expérience (MLFlow) :
 - Lancement du serveur MLFlow local pour tracker les métriques (Perte, Précision).
 - Intégration de la librairie codecarbon pour mesurer la consommation électrique réelle du GPU pendant l'entraînement.
- Entraînement du Challenger (Llama 3.2 3B) :
 - Script de Fine-tuning utilisant PEFT (LoRA) pour adapter le modèle génératif à la classification.
 - Paramétrage spécifique pour PyTorch sur ROCm (device='cuda').
 - Enregistrement des logs et du modèle final.
- Entraînement du Champion (DeBERTa-v3-base) :
 - Script de Fine-tuning classique utilisant transformers.Trainer.
 - Optimisation des hyperparamètres pour la classification de séquence.
 - Enregistrement des logs et du modèle final.
- Benchmark Final & Sélection :
 - Exécution du script de comparaison sur le jeu de test.
 - Analyse des graphiques MLFlow : Comparaison Précision vs Latence vs CO2.
 - Sélection du modèle vainqueur pour la production.

3.4 Validation & Packaging (Qualité Modèle)

- Tests Automatisés du Modèle (Deepchecks) :
 - Écriture d'une suite de tests pour vérifier l'intégrité du modèle (Data Leakage, Biais, Robustesse au bruit).
 - Génération d'un rapport de validation automatique.
- Packaging pour Inférence :
 - Conversion/Sauvegarde du modèle gagnant dans un format optimisé (ex: safetensors ou ONNX si applicable).
 - Push du modèle validé via DVC vers le stockage partagé.
 - Rédaction de la "Model Card" (Documentation du modèle : données utilisées, limites, métriques).

3.5 Déploiement MLOps (Monitoring)

- Définition des Métriques de Production :
 - Identification des indicateurs clés à surveiller (Drift des données, Temps de réponse, Pourcentage de classification "Green").
- Configuration du Monitoring (Prometheus) :
 - Préparation des exporteurs pour envoyer les métriques d'inférence vers Prometheus.
 - (L'intégration effective se fera lors du développement de l'API à l'étape

suivante).

ÉTAPE 4 : Backend & API (Blocs E1 & E4)

Cette étape consiste à construire le cœur fonctionnel de l'application : une API REST sécurisée qui expose les données et les fonctionnalités d'IA.

4.1 Conception de l'Architecture API (Design)

- Définition des Endpoints (Spécification OpenAPI) :
 - Identification des routes nécessaires :
 - GET /articles : Liste des articles analysés (pagination).
 - GET /articles/{id} : Détails d'un article.
 - POST /analyze : Point d'entrée pour l'analyse IA (URL ou Texte).
 - GET /stats : Statistiques globales (Ratio Green IT).
 - POST /auth/login : Authentification.
- Sécurisation (OWASP) :
 - Adoption du standard OAuth2 avec jetons JWT (JSON Web Tokens) pour l'authentification.
 - Définition des règles de validation des entrées (Input Validation) via Pydantic pour éviter les injections.

4.2 Développement du Serveur API (FastAPI)

- Initialisation du Projet FastAPI :
 - Configuration de l'application principale (titre, version, description).
 - Configuration des règles CORS (Cross-Origin Resource Sharing) pour autoriser le futur Frontend React.
- Connexion Base de Données (SQLAlchemy + Asyncpg) :
 - Intégration de la connexion asynchrone à PostgreSQL configurée à l'étape 2.
 - Création des modèles Pydantic (Schémas) pour sérialiser les réponses JSON.
- Mise en place des Logs (Loguru) :
 - Configuration du logger pour remplacer les print() par des logs structurés (INFO, ERROR, DEBUG).
 - Interception des logs système Uvicorn pour centralisation.

4.3 Implémentation de la Sécurité (Auth)

- Gestion Utilisateurs (FastAPI Users) :
 - Configuration du gestionnaire d'authentification (Bearer Transport).
 - Implémentation des stratégies de création de compte et de login.
 - Connexion de la table User PostgreSQL via l'ORM SQLAlchemy.
- Protection des Routes :
 - Ajout de dépendances de sécurité sur les routes sensibles (ex: POST /analyze nécessite d'être connecté).

4.4 Exposition des Données (Bloc E1 - C5)

- Développement des Endpoints de Lecture :
 - Implémentation de la logique de récupération des articles depuis la base de données.
 - Ajout des filtres (par date, par statut "Green IT").
 - Gestion de la pagination pour ne pas surcharger le client.

4.5 Intégration de l'IA dans l'API (Bloc E3 - C9/C10)

- Endpoint d'Analyse (POST /analyze) - Le Cœur du Réacteur :
 - Développement de la logique d'orchestration :
 1. Réception de l'URL ou du texte.
 2. Déclenchement du Scraping (si URL) via le module créé à l'étape 2.
 3. Déclenchement du nettoyage.
 4. Chargement du Modèle IA Custom (DeBERTa ou Llama) pour inférence.
 5. Appel de l'API SaaS Hugging Face pour le résumé.
 6. Agrégation des résultats et sauvegarde en base.
 - Optimisation : Chargement du modèle IA au démarrage de l'API (pour éviter de le recharger à chaque requête).

4.6 Documentation & Tests d'Intégration

- Documentation Automatique (Swagger UI / ReDoc) :
 - Vérification de la génération automatique de la documentation interactive sur /docs.
 - Ajout de descriptions et d'exemples pour chaque endpoint.
- Tests API (Pytest + HTTPX) :
 - Écriture de tests d'intégration pour vérifier que les endpoints répondent correctement (Code 200).
 - Test des cas d'erreur (Code 401 Unauthorized, Code 422 Validation Error).
 - Vérification de la chaîne complète (Input -> BDD -> Output).



ÉTAPE 5 : Frontend & Application (Bloc E4)

Cette étape concerne la réalisation de l'interface utilisateur (Application Cliente) en utilisant React et Shadcn/UI, avec un focus fort sur l'accessibilité (Audit Axe-core).

5.1 Initialisation de l'environnement Frontend (React & Vite)

- Création du projet :
 - Génération du squelette de l'application via Vite avec le template React + TypeScript.
 - Nettoyage des fichiers par défaut pour partir sur une base propre.
- Configuration du Styling (Tailwind CSS) :
 - Installation et initialisation de Tailwind CSS.
 - Configuration du fichier `tailwind.config.js` pour définir les chemins des contenus.
 - Ajout des directives Tailwind dans le fichier CSS principal.
- Installation du Design System (Shadcn/UI) :
 - Initialisation de la CLI Shadcn/UI pour configurer les variables CSS globales (thème, radius).
 - Choix d'un style neutre (Slate ou Zinc) pour un rendu professionnel "Green IT".
 - Installation de la bibliothèque d'icônes `lucide-react`.

5.2 Développement des Composants UI (Design System)

- Installation des composants Shadcn nécessaires :
 - Ajout des composants de base via la CLI : `button`, `input`, `card` (pour les articles), `badge` (pour le statut Green/Red), `table` (pour la liste), `dialog` (pour les alertes), `form` (gestion formulaires), `toast` (notifications).
- Création du Layout Principal :
 - Développement d'un composant de mise en page global incluant :
 - Une barre de navigation (Header) responsive.
 - Un pied de page (Footer) avec mentions légales.
 - Un conteneur principal centré et responsive.

5.3 Développement des Pages & Parcours Utilisateur

- Page d'Authentification (Login) :
 - Création d'un formulaire de connexion sécurisé.
 - Intégration de la logique de stockage du Token JWT (`localStorage` ou `Cookie`).
 - Redirection automatique après connexion réussie.
- Page Dashboard (Accueil) :
 - Développement de la zone d'action principale : Champ de saisie (URL/Texte) + Bouton "Analyser".
 - Développement de la section "Dernières Analyses" affichant les articles sous

forme de cartes ou tableau.

- Intégration de graphiques statistiques (ex: Camembert "Ratio Green IT") via une librairie de chart compatible.
- Page Détail Article :
 - Affichage complet du résultat de l'analyse IA.
 - Mise en valeur visuelle du statut (Vert = Green IT, Rouge = Non Green).
 - Affichage du résumé généré par l'IA SaaS.
 - Affichage du score de confiance du modèle IA.

5.4 Intégration de la Logique Métier (API & State)

- Connexion Backend (Client HTTP) :
 - Configuration d'une instance fetch ou axios avec l'URL de base de l'API FastAPI.
 - Mise en place d'intercepteurs pour injecter automatiquement le Token d'authentification dans les headers.
- Gestion de l'État et des Requêtes :
 - Utilisation de Hooks (useEffect, useState) ou d'une librairie (React Query) pour récupérer les données de l'API.
 - Gestion des états de chargement (Skeletons/Spinners) pendant l'analyse IA.
 - Gestion des erreurs (affichage de "Toasts" en cas d'échec serveur).

5.5 Tests & Accessibilité (Bloc E4 - A8)

- Audit d'Accessibilité Automatisé (Axe-core) :
 - Installation de la bibliothèque @axe-core/playwright.
 - Écriture d'un script de test Playwright qui navigue sur chaque page de l'application locale.
 - Injection du moteur Axe pour scanner la page et détecter les violations WCAG (Contraste couleurs, Labels manquants, Structure HTML).
 - Génération d'un rapport de conformité accessibilité.
- Vérification Responsive & UX :
 - Test manuel de l'interface sur différentes résolutions (Mobile, Tablette, Desktop).
 - Vérification de la navigation au clavier (Tabulation) pour tous les éléments interactifs.

ÉTAPE 6 : DevOps, Déploiement & Maintenance (Bloc E5)

Cette étape finale vise à industrialiser le projet en automatisant les tests, le déploiement, et en mettant en place une surveillance proactive de l'application en production.

6.1 Automatisation des Tests (CI Pipeline)

- Configuration de GitHub Actions :
 - Création du fichier de workflow YAML (.github/workflows/ci.yml).
 - Définition des déclencheurs (push sur main, pull request).
- Intégration des Étapes de Contrôle Qualité :
 - Étape de Linting : Exécution automatique de ruff pour vérifier le style du code.
 - Étape de Tests Unitaires/Intégration : Exécution de pytest pour valider le Backend.
 - Étape de Tests IA : Exécution de deepchecks pour valider la robustesse du modèle.
 - Étape de Tests Accessibilité : Exécution des tests Playwright/Axe-core pour valider le Frontend.

6.2 Conteneurisation & Packaging (Docker)

- Dockerisation du Backend (API) :
 - Rédaction du Dockerfile pour l'API Python.
 - Optimisation via "Multi-stage build" pour réduire la taille de l'image finale.
- Dockerisation du Frontend (React) :
 - Rédaction du Dockerfile pour l'application React.
 - Configuration du serveur web léger (NGINX ou module serve) pour distribuer les fichiers statiques buildés.
- Orchestration Locale (Docker Compose) :
 - Finalisation du fichier docker-compose.yml pour lancer toute la stack en une commande (API + Front + BDD + MinIO + Monitoring).

6.3 Livraison Continue (CD Pipeline)

- Configuration de l'Hébergement (Render) :
 - Liaison du compte Render au dépôt GitHub.
 - Création d'un "Web Service" pour l'API Docker.
 - Création d'un "Static Site" pour le Frontend React.
- Automatisation du Déploiement :
 - Configuration du déclenchement automatique du déploiement à chaque push validé sur la branche main.

- Vérification de la disponibilité de l'application en ligne (URL publique).

6.4 Monitoring & Observabilité (La Tour de Contrôle)

- Collecte de Métriques (Prometheus) :
 - Configuration finale de Prometheus pour "scraper" les métriques exposées par l'API FastAPI (/metrics).
 - Surveillance des indicateurs clés : CPU, Mémoire, Latence des requêtes HTTP, Nombre d'erreurs 500.
- Centralisation des Logs (Loki) :
 - Configuration du pilote de logging Docker pour envoyer les logs des conteneurs vers Loki.
 - Vérification de la remontée des logs structurés (JSON) générés par Loguru.
- Tableaux de Bord (Grafana) :
 - Création d'un Dashboard "Performance Système" (Utilisation ressources).
 - Création d'un Dashboard "Métier GreenTech" (Nombre d'articles analysés, Ratio Green/Non-Green, Temps moyen d'analyse IA).
 - Configuration d'alertes visuelles (ex: seuil rouge si latence > 2s).

6.5 Maintenance & Gestion d'Incidents (A9)

- Simulation d'Incidents (Chaos Engineering léger) :
 - Test de coupure volontaire de la base de données ou de l'API externe Hugging Face.
 - Vérification que l'application ne crashe pas brutalement et affiche un message d'erreur utilisateur compréhensible.
- Documentation de Maintenance :
 - Rédaction d'une procédure de débogage (Playbook) expliquant comment lire les logs dans Grafana pour identifier une erreur.
 - Documentation de la procédure de mise à jour du modèle IA en production sans interruption de service.